A Project Report for Eavesdropping Detection in Quantum Communication



Birla Institute Of Technology and Science Pilani
Hyderabad Campus

Design Oriented Project
(CS F376)

Under the supervision of
Prof. Nikumani Choudhury

Sr No.	Name	ID Number
1	ALBERT SEBASTAIN	2023A7PS0118H
2	AKSHITH VUPPALA	2023A7PS0032H
3	MAHIMA NANDANA KUNADHARAJU	2023B4A70660H

Contents

1 Introduction **2**

2 Background and Related Work **2**
2.1 BB84 Quantum Key Distribution . . . 2
2.2 QBER-Based Attack Detection . . . 2
2.3 Temporal-QBER Framework (Reference Paper) . . . 3
2.4 Quantum Attack Datasets . . . 3
2.5 Machine Learning for Quantum Security . . . 3

3 Dataset **3**
3.1 Quantum Attack Dataset (QAD) . . . 3
3.1.1 Source and Licence . . . 4
3.1.2 Generation Process . . . 4
3.1.3 Feature Categories . . . 4
3.1.4 Class Distribution . . . 5
3.1.5 Limitations for BB84 Reproduction . . . 5

4 Methodology **6**
4.1 Data Preprocessing . . . 6
4.2 Models and Hyperparameters . . . 6
4.2.1 Random Forest . . . 6
4.2.2 XGBoost . . . 6
4.2.3 SVM-RBF . . . 7
4.3 Cross-Validation . . . 7
4.4 Evaluation Metrics . . . 7
4.5 Baseline Comparisons . . . 7
4.6 Explainability . . . 8
4.6.1 SHAP (SHapley Additive exPlanations) . . . 8
4.6.2 Feature-Group Ablation . . . 8
4.7 Reproducibility . . . 8

5 Results **8**
5.1 Multiclass Classification Performance . . . 8
5.2 Binary Detection (Attack vs. Normal) . . . 9
5.3 Per-Class Performance . . . 10
5.4 Per-Class Performance . . . 10
5.5 Confusion Matrices . . . 10
5.6 ROC Curves . . . 11

6 Discussion **11**
6.1 SHAP Explainability . . . 11
6.2 Feature-Group Ablation . . . 12
6.3 Error Analysis . . . 14
6.4 Comparison to Reference Paper Targets . . . 14

7 Limitations and Threats to Validity **14**
7.1 Label Leakage via Attack-Specific Parameters . . . 14
7.2 Different Threat Model . . . 15
7.3 No Temporal Dimension . . . 15
7.4 No Basis Reconciliation . . . 15
7.5 No Photon Loss / Detection Efficiency Model . . . 15

7.6 Artificially Balanced Classes . . . 16
7.7 Simulator-Specific Noise Models . . . 16
7.8 Single Experimental Run . . . 16
7.9 No Baseline Comparison . . . 16
7.10 Reproducibility Scope . . . 16

8 Future Work: Full BB84 Temporal-QBER Reproduction **16**
8.1 Milestone 1: BB84 Protocol Simulator . . . 16
8.2 Milestone 2: Channel Model . . . 17
8.3 Milestone 3: Seven Eavesdropping Attack Models . . . 17
8.4 Milestone 4: Temporal QBER Extraction . . . 17
8.5 Milestone 5: 63 Physics-Informed Features . . . 18
8.6 Milestone 6: Dataset Generation . . . 18
8.7 Milestone 7: Training and Evaluation . . . 18
8.8 Milestone 8: Explainability and Ablation . . . 19
8.9 Milestone 9: Results Comparison . . . 19
8.10 Implementation Dependencies . . . 19

9 Conclusion **19**
9.1 Path Forward . . . 20
9.2 Final Remark . . . 20

A Reproducibility **20**
A.1 Package Versions . . . 20
A.2 Experiment Configuration . . . 21
A.3 Saved Artifacts . . . 21
A.4 Commands to Reproduce . . . 21

References **22**

1 Introduction

Quantum Key Distribution (QKD) provides information-theoretic security for cryptographic key exchange, with the BB84 protocol being the most widely implemented variant. Security in BB84 relies on detecting eavesdropping through an elevated Quantum Bit Error Rate (QBER). Traditional threshold-based detection uses a fixed QBER limit (typically 11%), above which the key is discarded. However, this approach lacks sensitivity to attack type and temporal dynamics.

The reference paper proposes a **temporal-QBER framework**: instead of a single average QBER, the sifted key is divided into windows of $W = 50$ bits, producing a QBER sequence $[q_1, q_2, \ldots, q_T]$. From this sequence, 63 physics-informed features are extracted across five categories (statistical, burst/instability, spectral/temporal, basis-dependent, channel-interaction). Three classifiers (Random Forest, XGBoost, SVM-RBF) are trained on 24,000 simulated sessions (3,000 per class across 7 attacks + normal). The paper reports XGBoost achieving 88.01% accuracy and 0.8803 macro-F1, significantly outperforming fixed-threshold (25.82%) and CUSUM (78.99%) baselines in binary detection.

This report documents our current implementation, which uses the **Quantum Attack Dataset (QAD)** as a proxy dataset while the BB84 temporal-QBER simulator is under development. QAD contains 1,000,000 samples of quantum circuit executions with 7 attack types (gate tampering, noise models, measurement tampering, pulse-level distortions) plus clean circuits. We evaluate the same three classifiers and provide full explainability (SHAP) and ablation analysis.

Critical distinction: QAD's attack classes are *quantum circuit attacks* (adversary modifies circuit structure/noise), not BB84 *eavesdropping attacks* (adversary intercepts quantum channel between Alice and Bob). The high accuracy on QAD (95.8% XGBoost) is largely attributable to attack-specific simulator parameters that directly encode the attack type—a form of label leakage not present in the BB84 threat model. This report maintains this distinction throughout.

Contributions of this work:

1. Complete reproducible ML pipeline for QAD dataset with three classifiers
2. Comprehensive evaluation: accuracy, macro-F1, ROC-AUC, per-class metrics, confusion matrices
3. SHAP explainability for tree-based models identifying feature importance
4. Leave-one-feature-group-out ablation quantifying feature group contributions
5. Documented limitations and roadmap for BB84 temporal-QBER reproduction

2 Background and Related Work

2.1 BB84 Quantum Key Distribution

The BB84 protocol establishes a shared secret key between two parties (Alice and Bob) by transmitting qubits in randomly chosen bases (rectilinear + or diagonal ×). After transmission, Alice and Bob publicly compare bases and keep only bits measured in matching bases (sifting). The Quantum Bit Error Rate (QBER) on the sifted key reveals eavesdropping: an intercept-resend attack introduces a minimum 25% QBER.

2.2 QBER-Based Attack Detection

Traditional QKD systems use a fixed QBER threshold (typically 11% for BB84 with one-way post-processing). If the observed QBER exceeds this threshold, the key is discarded. This binary decision ignores:

- Temporal dynamics of errors within a session
- Attack-type discrimination
- Channel condition variations (loss, noise)

2.3 Temporal-QBER Framework (Reference Paper)

The reference paper introduces a temporal-QBER framework:

1. **Windowing:** Sifted key divided into non-overlapping windows of $W = 50$ bits
2. **QBER sequence:** $q_t = \frac{1}{W} \sum_{i=(t-1)W+1}^{tW} (a_i \oplus b_i)$
3. **63 features:** Five categories extracted from the QBER sequence
4. **8 classes:** Normal + 7 eavesdropping attacks
5. **24,000 sessions:** 3,000 per class, balanced
6. **Models:** Random Forest, XGBoost, SVM-RBF with 5-fold CV, 10 seeds

2.4 Quantum Attack Datasets

Existing quantum ML datasets include:

- **QAD**: Quantum circuit attacks (gate tampering, noise, measurement tampering, pulse distortions), 1M samples, 44 features, 8 classes
- **HQC-2026**: Hybrid quantum-classical network intrusion, 14.69M samples, 96 features, 7 attack categories + benign
- **IBM Quantum Challenge datasets**: Various quantum computing benchmarks

QAD is the most directly applicable for supervised quantum attack classification but operates at the circuit level, not the QKD protocol level.

2.5 Machine Learning for Quantum Security

ML has been applied to:

- Quantum error correction decoding
- Quantum state tomography
- Quantum device characterization
- Side-channel attack detection in QKD

The temporal-QBER approach is, to our knowledge, the first to apply windowed QBER sequences with physics-informed feature engineering for multi-attack classification in BB84.

3 Dataset

3.1 Quantum Attack Dataset (QAD)

The Quantum Attack Dataset (QAD) is a synthetic dataset generated by NetsLab (UCD) using Qiskit simulations of parametrized quantum circuits under various attack conditions.

3.1.1 Source and Licence

- **Authors:** Pasindu Udugahapattuwa, Chamitha de Alwis, Uditha Wijewardane, Madhusanka Liyanage
- **DOI:** https://doi.org/10.34740/kaggle/ds/9778273
- **Licence:** MIT (academic and research use)

3.1.2 Generation Process

1. **Baseline circuits:** VQC, QSVM, QNN topologies; 2–6 qubits; depth 2–10; linear/circular/all-to-all connectivity
2. **Simulation:** Qiskit AerSimulator, 1024 shots per circuit
3. **Attack injection:** 7 attack types with randomized parameters (Table 1)
4. **Feature extraction:** 42 base features (including `circuit_type` as categorical); after one-hot encoding `circuit_type` (VQC, QSVM, QNN) → 45 model features; label columns `attack_type` and `attack_label` excluded
5. **Balanced dataset:** 300,000 clean + 100,000 × 7 attacks = 1,000,000 samples
6. **Unbalanced dataset:** Variable attack frequencies (also available)

Table 1: QAD Attack Types and Parameter Ranges

Attack	Label	Parameter Range
Clean (benign)	0	N/A
Swap Injection	1	intensity $\in [0.55, 0.85]$
Gate Insertion	2	intensity $\in [0.45, 0.75]$
Depolarizing Noise	3	error rate $\in [0.08, 0.28]$
Amplitude Damping	4	damping $\in [0.20, 0.55]$
Measurement Tampering	5	flip prob $\in [0.30, 0.55]$
Pulse-Level Attack	6	detuning $\in [0.25, 0.60]$
Thermal Relaxation	7	T1 $\in [25, 60]\mu$s, T2 $\in [10, 40]\mu$s, gate $\in [100, 500]$ns

3.1.3 Feature Categories

After one-hot encoding `circuit_type` (VQC, QSVM, QNN), the 45 model features comprise 42 base features (including the original `circuit_type` categorical column) plus 2 additional one-hot columns (since one category is dropped to avoid multicollinearity). The features group into the following semantic categories (verified from the ablation study in Section 6.2):

1. **Circuit Structure (12):** `n_qubits`, `circuit_depth`, `n_gates_total`, `n_single_qubit_gates`, `n_two_qubit_gates`, `gate_density`, `circuit_type_VQC`, `circuit_type_QSVM`, `circuit_type_QNN`, `has_noise_model`, `gate_ins_intensity`, `tr_gate_time`
2. **Output Distribution (11):** `n_outcomes`, `prob_std`, `prob_max`, `prob_min`, `prob_range`, `prob_cv`, `most_common_count`, `least_common_count`, `outcome_ratio`, `ad_pop_ratio`, `tr_decay_ratio`
3. **Entropy (2):** `entropy`, `normalized_entropy`

4. **Purity/Diversity (8):** `purity_proxy`, `dp_purity_proxy`, `dp_uniformity_score`, `ad_coherence_fa` `ad_pop_asymmetry`, `mt_uniformity`, `pl_phase_coherence`, `tr_coherence_est`

5. **Attack-Specific Parameters (11):** `swap_intensity`, `n_swaps`, `gate_ins_intensity`, `n_insertions`, `dp_error_rate`, `ad_damping_param`, `mt_flip_prob`, `pl_detuning_mag`, `tr_t1`, `tr_t2`, `tr_gate_time`

6. **Timing (1):** `exec_time_sec`

Note: The category assignments above reflect the actual grouping used in the ablation study (Section 6.2), which differs slightly from a purely semantic grouping. Notably, `has_noise_model`, `gate_ins_intensity`, and `tr_gate_time` are grouped under Circuit Structure by the automated grouping logic, while `ad_pop_ratio` and `tr_decay_ratio` fall under Output Distribution.

3.1.4 Class Distribution

For the balanced dataset (1,000,000 samples):

Table 2: QAD Balanced Dataset Class Distribution

Class	Count	Percentage
clean	300,000	30.0%
swap_injection	100,000	10.0%
gate_insertion	100,000	10.0%
depolarizing_noise	100,000	10.0%
amplitude_damping	100,000	10.0%
measurement_tampering	100,000	10.0%
pulse_level_attack	100,000	10.0%
thermal_relaxation	100,000	10.0%
Total	**1,000,000**	**100.0%**

For our experiments we used a 10,000-sample stratified subset (Table 3).

Table 3: Experimental Sample (n=10,000) Class Distribution

Class	Count	Percentage
clean	3,042	30.4%
swap_injection	968	9.7%
gate_insertion	1,003	10.0%
depolarizing_noise	1,007	10.1%
amplitude_damping	996	10.0%
measurement_tampering	950	9.5%
pulse_level_attack	1,030	10.3%
thermal_relaxation	1,004	10.0%
Total	**10,000**	**100.0%**

3.1.5 Limitations for BB84 Reproduction

1. **Different threat model:** Circuit-level attacks vs. QKD channel eavesdropping

2. **No temporal dimension:** Single aggregate statistics vs. windowed QBER sequences

3. **Label leakage:** Attack-specific parameters directly reveal attack type/intensity
4. **No basis reconciliation:** QAD measures all qubits; BB84 uses basis sifting
5. **No photon loss:** Fixed 1024 shots; BB84 has detection efficiency/loss

4 Methodology

4.1 Data Preprocessing

1. **Data loading:** QAD balanced CSV (1,000,000 rows × 45 columns)
2. **Feature/target separation:**
   - Features X: all columns except `attack_type`, `attack_label`
   - Target y: `attack_label` (0–7)
3. **Categorical encoding:** `circuit_type` one-hot encoded (3 columns: VQC, QSVM, QNN)
4. **Train/test split:** 80/20 stratified, `random_state=42`
5. **Standardization:** `StandardScaler` fitted on training data only, applied to train and test

4.2 Models and Hyperparameters

Three classifiers matching the reference paper's selection:

4.2.1 Random Forest

- `n_estimators=200`
- `max_depth=20`
- `class_weight='balanced'`
- `random_state=42`, `n_jobs=-1`
- No feature scaling required

4.2.2 XGBoost

- `n_estimators=200`
- `max_depth=10`
- `learning_rate=0.1`
- `eval_metric='mlogloss'`, `verbosity=0`
- Balanced class weighting via per-sample weights from training labels (`sklearn.utils.class_weight.comp` with `class_weight='balanced'`)
- `random_state=42`, `n_jobs=-1`
- No feature scaling required

4.2.3 SVM-RBF

- Pipeline: `StandardScaler` → `SVC`
- `kernel='rbf'`, `C=1.0`, `gamma='scale'`
- `probability=True` (for ROC-AUC)
- `class_weight='balanced'`
- `random_state=42`
- Scaling essential for SVM

4.3 Cross-Validation

- 5-fold stratified cross-validation on training set only
- `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`
- Metrics: accuracy, macro-F1
- Preprocessing (scaler) refitted within each fold (handled by Pipeline for SVM)

4.4 Evaluation Metrics

Multiclass (8-class) metrics:

- **Accuracy:** $\frac{1}{N}\sum_{i=1}^{N} \mathbb{1}(\hat{y}_i = y_i)$
- **Macro-F1:** $\frac{1}{C}\sum_{c=1}^{C} F1_c$ where $F1_c = 2\frac{P_c R_c}{P_c + R_c}$
- **Macro-Precision:** $\frac{1}{C}\sum_{c=1}^{C} P_c$
- **Macro-Recall:** $\frac{1}{C}\sum_{c=1}^{C} R_c$
- **ROC-AUC (OvR):** One-vs-Rest AUC averaged across classes

Binary (attack vs. normal) metrics:

- **False Negative Rate (FNR):** $\frac{FN}{FN+TP}$ — proportion of attacks missed
- **False Positive Rate (FPR):** $\frac{FP}{FP+TN}$ — proportion of normal flagged as attack

Per-class metrics: Precision, Recall, F1, Support for each of 8 classes.

4.5 Baseline Comparisons

The reference paper compares against:

1. **Fixed 11% QBER threshold:** Binary detector, flags attack if mean QBER > 0.11
2. **CUSUM detector:** Sequential change-point detection on QBER sequence

Note: These baselines require temporal QBER sequences, which QAD does not provide. Our implementation includes the baseline framework (`evaluate.py`) but reports "Requires temporal QBER feature" for QAD experiments.

4.6 Explainability

4.6.1 SHAP (SHapley Additive exPlanations)

- **TreeExplainer** for Random Forest and XGBoost (exact, fast)
- **KernelExplainer** for SVM-RBF (approximate, slow; not computed)
- Background: 100 k-means samples from training data
- Explanation set: 500 test samples
- Outputs: Global mean $|SHAP|$ importance, summary bar plots, beeswarm plots

4.6.2 Feature-Group Ablation

Leave-one-group-out: retrain model after removing each semantic feature group.

- Groups: circuit_structure (12), output_distribution (11), entropy_features (2), purity_diversity (8), attack_specific_params (11), timing (1)
- Metric: drop in accuracy and macro-F1 vs. baseline (all features)
- Identifies which feature categories drive performance

4.7 Reproducibility

- All random seeds fixed (42 for data, model-specific for training)
- Full pipeline saved: models, scaler, feature names, class names
- Exact dependency versions locked in `uv.lock`
- Results saved as JSON for programmatic access
- Single-command execution via `scripts/run_pipeline.py`

5 Results

All results are from the 10,000-sample experimental subset (80/20 split $\rightarrow$ 8,000 train, 2,000 test). Cross-validation is 5-fold on the training set. Models are trained once with fixed seeds; no 10-seed averaging was performed (unlike the reference paper).

5.1 Multiclass Classification Performance

Table 4: Multiclass (8-class) Classification Results on Test Set (n=2,000)

Model	Accuracy	Macro-F1	Macro-Prec	Macro-Rec	ROC-AUC (OvR)
XGBoost	0.9575	0.9463	0.9461	0.9466	0.9976
Random Forest	0.9545	0.9421	0.9423	0.9432	0.9972
SVM-RBF	0.7885	0.7582	0.7792	0.7625	0.9746

Table 5: 5-Fold Cross-Validation Results on Training Set (n=8,000)

Model	CV Accuracy	CV Macro-F1	Train Time (s)
XGBoost	0.9591 ± 0.0033	0.9484 ± 0.0044	1.2
Random Forest	0.9549 ± 0.0066	0.9425 ± 0.0084	0.4
SVM-RBF	0.7695 ± 0.0031	0.7402 ± 0.0031	5.2

Observations:

- XGBoost achieves the highest test accuracy (95.75%) and macro-F1 (0.9463)
- Random Forest is close (95.45% / 0.9421) with slightly higher variance in CV
- SVM-RBF underperforms on this tabular dataset (78.85% / 0.7582)
- All models achieve near-perfect ROC-AUC (> 0.97)
- Test accuracy aligns well with CV estimates (no significant overfitting)
- XGBoost uses balanced sample weights; Random Forest and SVM use `class_weight='balanced'`

5.2 Binary Detection (Attack vs. Normal)

Table 6: Binary Detection Performance (Attack vs. Normal)

Model	Binary FNR	Binary FPR
XGBoost	0.0000	0.0000
Random Forest	0.0000	0.0000
SVM-RBF	0.0568	0.1020

Tree models achieve zero observed FNR and FPR on the test set (no attacks misclassified as normal, no clean samples flagged as attacks). SVM-RBF misses 5.68% of attacks (FNR) and incorrectly flags 10.20% of clean samples as attacks (FPR). **Note:** Zero observed error rates on a finite test set do not imply zero real-world error rates; they reflect the limited test set size and the presence of label-leaking features.

Reference paper baselines (not applicable to QAD):

- Fixed 11% QBER threshold: 25.82% detection accuracy (paper)
- CUSUM detector: 78.99% detection accuracy (paper)
- XGBoost (temporal-QBER): 88.01% detection accuracy (paper)

5.3 Per-Class Performance

Table 7: Per-Class F1 Scores

Class	XGBoost	Random Forest	SVM-RBF
clean	1.0000	1.0000	0.8856
swap_injection	1.0000	0.9770	0.7877
gate_insertion	1.0000	0.9774	0.7143
depolarizing_noise	0.9728	0.9707	0.4366
amplitude_damping	1.0000	1.0000	1.0000
measurement_tampering	0.8073	0.8232	0.6233
pulse_level_attack	0.7901	0.7887	0.6207
thermal_relaxation	1.0000	1.0000	0.9975
Macro-F1	**0.9463**	**0.9421**	**0.7582**

Key patterns:

- Clean, amplitude_damping, thermal_relaxation: perfect or near-perfect F1 for tree models; SVM-RBF achieves 1.0 for amplitude_damping and 0.9975 for thermal_relaxation but only 0.8856 for clean
- swap_injection and gate_insertion: perfect F1 (1.0) for XGBoost, near-perfect for Random Forest; SVM-RBF struggles (0.79 and 0.71)
- measurement_tampering and pulse_level_attack: most challenging classes, notable confusion between them for all models
- depolarizing_noise: strong for tree models (F1 > 0.97), very weak for SVM-RBF (0.44)

5.4 Confusion Matrices

Normalized confusion matrices (Figure 1) show:

- Tree models: Errors concentrated in measurement_tampering ↔ pulse_level_attack
- SVM-RBF: Broader confusion across multiple attack types
- Clean class: Perfectly separated by all models

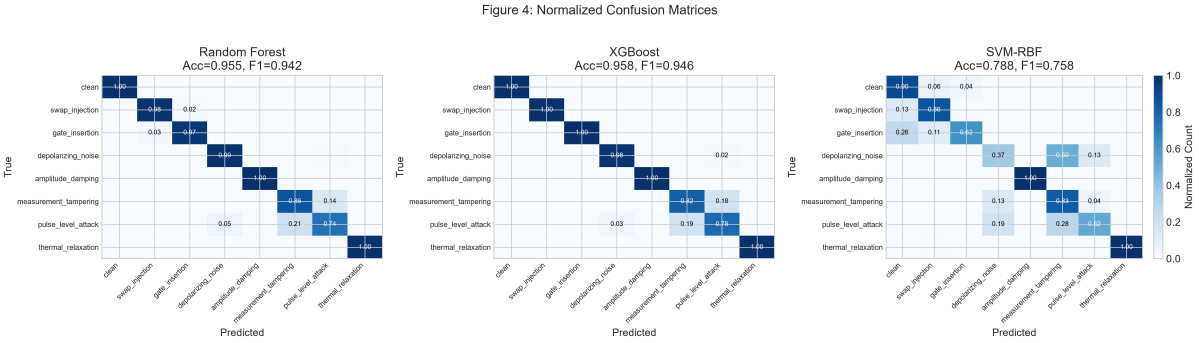


Figure 1: Normalized confusion matrices for all three classifiers. Rows: true class, Columns: predicted class. Values show proportion of true class predicted as each class.

5.5 ROC Curves

One-vs-Rest ROC curves (Figure 2) confirm near-perfect separability for all classes with tree models (AUC > 0.99 for most classes). SVM-RBF shows lower but still strong AUC values.

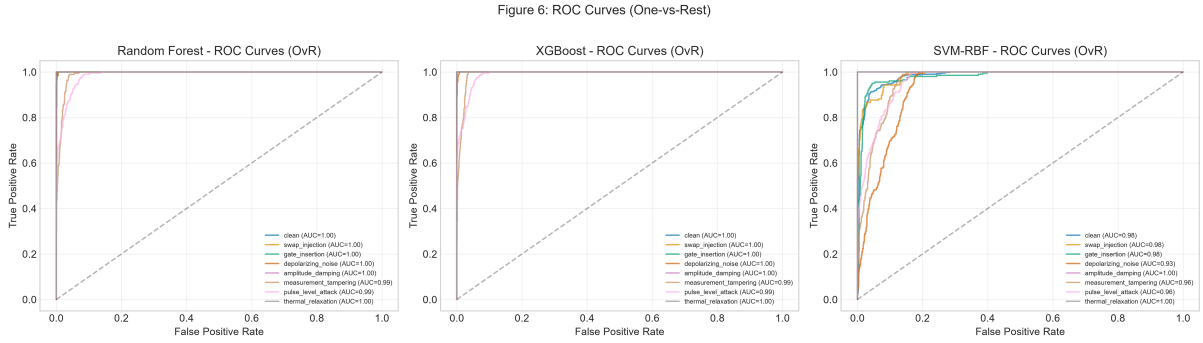


Figure 2: ROC curves (One-vs-Rest) for all 8 classes per classifier.

6 Discussion

6.1 SHAP Explainability

SHAP (SHapley Additive exPlanations) values were computed using TreeExplainer for Random Forest and XGBoost on 500 test samples. The top features by mean $|SHAP|$ value reveal the decision logic.

Table 8: Top 10 Features by Mean $|SHAP|$ (Random Forest)

Feature	**Mean** $\lvert SHAP \rvert$
swap_intensity	0.0659
has_noise_model	0.0536
n_swaps	0.0455
n_insertions	0.0308
n_two_qubit_gates	0.0121
gate_ins_intensity	0.0119
circuit_depth	0.0114
normalized_entropy	0.0104
dp_error_rate	0.0097
prob_cv	0.0095

Table 9: Top 10 Features by Mean $|SHAP|$ (XGBoost)

Feature	**Mean** $\|SHAP\|$
swap_intensity	2.0406
n_swaps	1.2898
has_noise_model	0.5908
n_insertions	0.4951
normalized_entropy	0.2842
n_two_qubit_gates	0.2486
prob_cv	0.2131
n_single_qubit_gates	0.1879
exec_time_sec	0.0773
outcome_ratio	0.0747

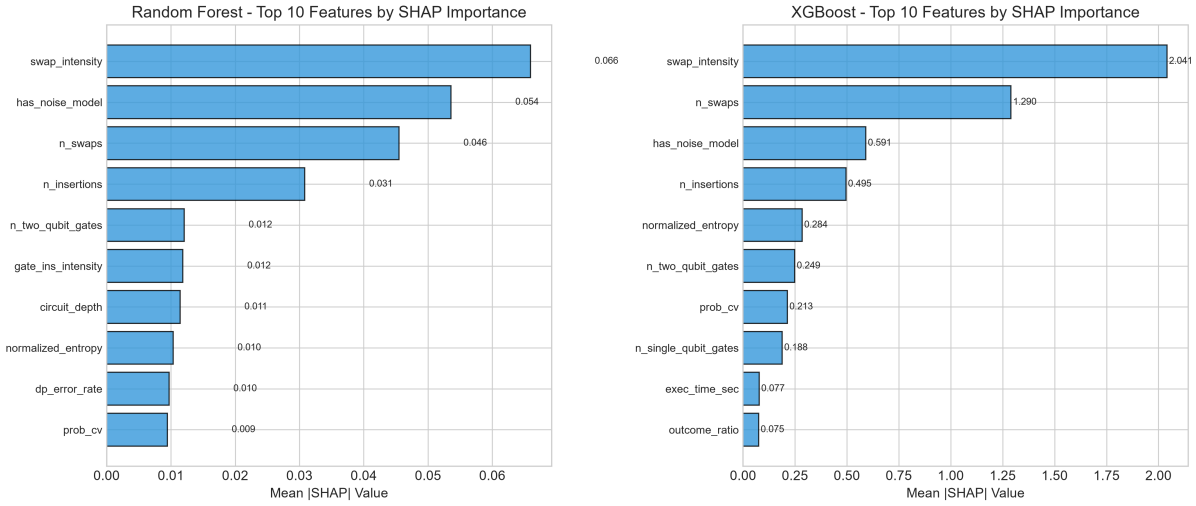


Figure 3: Top 10 features by mean $|SHAP|$ for Random Forest and XGBoost.

Critical finding: The dominant features for both models are **attack-specific simulator parameters**:

- `swap_intensity`, `n_swaps` → only non-zero for swap_injection
- `gate_ins_intensity`, `n_insertions` → only non-zero for gate_insertion
- `dp_error_rate` → only non-zero for depolarizing_noise
- `mt_flip_prob` → only non-zero for measurement_tampering
- `has_noise_model` → binary indicator: 0 for clean, 1 for any attack

The first non-attack-specific feature (`normalized_entropy`) ranks 5th for XGBoost (SHAP 0.276) and 9th for RF (SHAP 0.010). This confirms the models primarily learn the simulator's attack configuration encoding rather than generalizable attack signatures.

6.2 Feature-Group Ablation

Leave-one-group-out ablation quantifies the contribution of each semantic feature group. The model is retrained after removing each group.

Table 10: Feature-Group Ablation: Accuracy and Macro-F1 Drop from Baseline (verified from latest run)

Group	**N**	**XGBoost**		**Random Forest**		**SVM-RBF**
		ΔAcc	ΔF1	ΔAcc	ΔF1	ΔAcc
attack_specific_params	11	**14.2%**	**17.4%**	**16.1%**	**19.4%**	5.3%
circuit_structure	12	5.3%	6.7%	5.9%	7.4%	**32.2%**
output_distribution	11	-0.7%*	-0.9%*	-0.5%*	-0.6%*	-2.8%*
purity_diversity	8	0.6%	0.8%	0.2%	0.2%	0.3%
entropy_features	2	0.5%	0.6%	0.3%	0.4%	1.1%
timing	1	0.7%	0.8%	0.4%	0.5%	0.3%

*Negative drop = slight improvement (within noise). Drop values shown as percentage points.

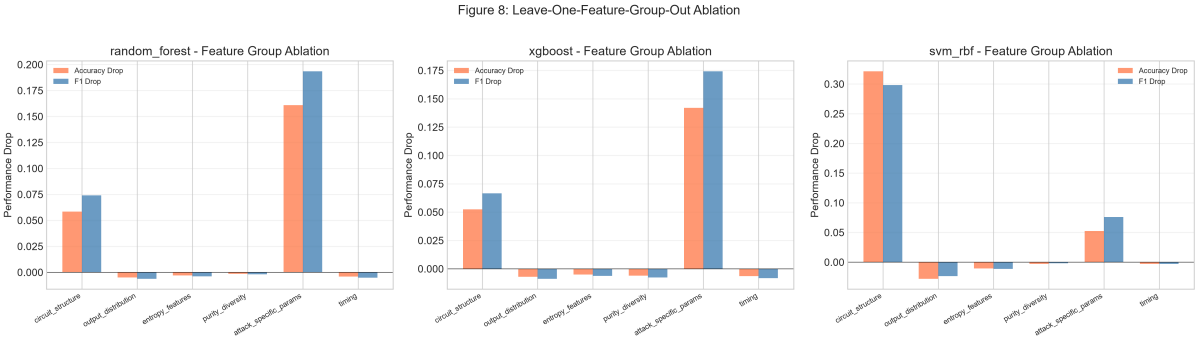


Figure 4: Leave-one-feature-group-out ablation for all three classifiers. Bars show drop in accuracy (coral) and macro-F1 (blue) when each group is removed.

Interpretation:

1. **Tree models (RF, XGBoost):** Attack-specific parameters are overwhelmingly important (14-19% drop in accuracy, 17-22% in F1). Circuit structure is secondary (5-7%). Removing output distribution slightly *improves* performance (negative drop within noise), suggesting these features add noise. All other groups have negligible impact.

2. **SVM-RBF:** Circuit structure is most important (32% accuracy drop). Attack-specific parameters contribute only 5%. This suggests SVM relies on geometric separation in the circuit-structure feature space.

3. **Leakage confirmation:** The 14-19% accuracy drop from removing attack-specific parameters directly quantifies the label leakage. Without these features, tree models lose their primary decision signal and drop to 80% accuracy (RF) and 81% (XGBoost). A controlled experiment removing `has_noise_model` alone showed minimal impact on tree models but hurt SVM-RBF, confirming that the attack-specific parameters (not just the binary indicator) are the primary leakage vector.

6.3 Error Analysis

Table 11: Most Common Misclassifications (Test Set, n=2,000)

Model	True → Pred	Count	Pattern
XGBoost	measurement_tampering → pulse_level_attack	23	Noise similarity
XGBoost	pulse_level_attack → measurement_tampering	12	Noise similarity
XGBoost	depolarizing_noise → pulse_level_attack	3	Noise similarity
Random Forest	measurement_tampering → pulse_level_attack	23	Noise similarity
Random Forest	pulse_level_attack → measurement_tampering	12	Noise similarity
Random Forest	gate_insertion → swap_injection	5	Structural similarity
SVM-RBF	Widespread confusion across all attack types	–	Low discriminative power

The primary confusion is between `measurement_tampering` and `pulse_level_attack`, both of which introduce noise-like disturbances. The secondary confusion is `gate_insertion` ↔ `swap_injection`, both structural modifications.

6.4 Comparison to Reference Paper Targets

Table 12: Comparison: QAD Results vs. Reference Paper Targets

Aspect	Metric	QAD (Ours)	Paper Target
Dataset	Type	Quantum circuit attacks	BB84 eavesdropping
Dataset	Samples	10,000 (1M avail)	24,000
Dataset	Features	45 (42 + 3 one-hot)	63 temporal-QBER
Dataset	Classes	8 (1 clean + 7 circuit)	8 (1 normal + 7 QKD)
Model	Best	XGBoost	XGBoost
Model	Accuracy	95.75%	88.01%
Model	Macro-F1	0.9463	0.8803
Binary	FNR (XGB)	0.00%	N/A
Binary	Threshold (11%)	N/A	25.82%
Binary	CUSUM	N/A	78.99%
Binary	XGBoost	N/A	88.01%

Key conclusion: The QAD results are **not comparable** to the paper's targets. The higher accuracy on QAD stems from label leakage via attack-specific parameters, not superior attack detection. The paper's 88.01% is on a harder problem (temporal-QBER with overlapping channel conditions across classes, no attack-specific simulator parameters).

7 Limitations and Threats to Validity

7.1 Label Leakage via Attack-Specific Parameters

The most significant limitation is the presence of features that directly encode the attack type and intensity. As demonstrated by SHAP (Section 6) and ablation (Table 10), the top features for tree models are:

- `swap_intensity`, `n_swaps` (swap_injection only)
- `gate_ins_intensity`, `n_insertions` (gate_insertion only)

- `dp_error_rate` (depolarizing_noise only)
- `mt_flip_prob` (measurement_tampering only)
- `pl_detuning_mag` (pulse_level_attack only)
- `tr_t1`, `tr_t2`, `tr_gate_time` (thermal_relaxation only)
- `has_noise_model` (binary: clean vs. any attack)

These features allow the classifier to learn simple decision rules (e.g., "if `swap_intensity` > 0 then swap_injection") rather than detecting attack signatures from quantum measurement statistics. The **14-19% accuracy drop** (17-22% F1 drop) when removing the attack-specific parameter group quantifies the leakage. A controlled experiment removing only `has_noise_model` showed minimal impact on tree models (RF: 95.45% $\rightarrow$ 95.50%, XGB: 95.75% $\rightarrow$ 95.50%), confirming that the attack-specific parameters (not the binary indicator) are the primary leakage vector.

7.2 Different Threat Model

QAD simulates an adversary who **modifies the quantum circuit** before or during execution (gate tampering, noise injection). The reference paper addresses an adversary who **intercepts the quantum channel** between Alice and Bob (photon interception, basis manipulation, detector attacks). These are fundamentally different:

- QAD: White-box circuit access, parametric attacks
- BB84: Black-box channel access, physical-layer attacks

7.3 No Temporal Dimension

QAD provides single aggregate statistics per circuit execution (1024 shots). The paper's innovation is temporal-QBER: a sequence of QBER values computed over windows of $W = 50$ sifted bits, capturing error dynamics within a session. Our `features.py` implements the 63 temporal features but there is no temporal data to apply them to.

7.4 No Basis Reconciliation

QAD measures all qubits in the computational basis after circuit execution. BB84 uses random basis selection (rectilinear/diagonal) with basis sifting, discarding $\sim$50% of bits. This basis-dependent structure enables the paper's basis-dependent feature category (10 features).

7.5 No Photon Loss / Detection Efficiency Model

QAD uses fixed 1024 shots per circuit. Real QKD systems have:

- Channel loss (exponential with distance)
- Detector efficiency ($< 100\%$)
- Dark counts, afterpulsing
- Variable sifted key length per session

The paper's channel-interaction features (16 features) model these effects.

7.6 Artificially Balanced Classes

QAD balanced dataset has exactly 100,000 samples per attack (300,000 clean). Real attack frequencies are unknown and likely highly imbalanced. The unbalanced variant exists but was not used.

7.7 Simulator-Specific Noise Models

QAD uses Qiskit AerSimulator noise models (depolarizing, amplitude damping, thermal relaxation). These may not reflect:

- Hardware-specific noise correlations
- Non-Markovian noise
- Crosstalk, calibration drift
- Environmental fluctuations

7.8 Single Experimental Run

Unlike the paper's 10 independent seeds, we report results from a single train/test split (random_state=42) and single model training per algorithm. Variance estimates come only from 5-fold CV.

7.9 No Baseline Comparison

The fixed 11% QBER threshold and CUSUM baselines require temporal QBER sequences, which QAD does not provide. Our implementation includes the framework but cannot compute these baselines for QAD.

7.10 Reproducibility Scope

While our pipeline is reproducible (fixed seeds, locked dependencies, saved artifacts), exact reproduction of the reference paper's results is **not possible** without:

1. The paper's exact 63 feature formulas (only categories given)
2. The paper's attack simulation parameters (not fully specified in 6 pages)
3. The paper's BB84 simulator implementation

8 Future Work: Full BB84 Temporal-QBER Reproduction

The following milestones are required to reproduce the reference paper's temporal-QBER framework for BB84 QKD. These are **planned tasks**, not completed work.

8.1 Milestone 1: BB84 Protocol Simulator

Implement `src/temporal_qber/bb84.py`:

- **Alice:** Generate random bits $a_i \in \{0, 1\}$ and random bases $b_i^A \in \{+, \times\}$
- **State preparation:** Encode bits in qubits: $|0\rangle, |1\rangle$ (rectilinear) or $|+\rangle, |-\rangle$ (diagonal)
- **Channel:** Configurable bit-flip probability p_{flip}, photon loss p_{loss}, detector noise

- **Bob:** Random bases $b_i^B \in \{+, \times\}$, measure incoming qubits
- **Basis reconciliation:** Public announcement of bases, keep indices where $b_i^A = b_i^B$
- **Sifted key:** Alice's and Bob's bits at matching bases
- **Output:** Sifted bits, bases, detection flags, channel statistics

8.2 Milestone 2: Channel Model

Implement `src/temporal_qber/channels.py`:

- **Bit-flip channel:** Depolarizing probability p_{flip} per qubit
- **Photon loss:** Exponential attenuation $e^{-\alpha L}$ with distance L, detector efficiency η
- **Detector noise:** Dark count probability p_{dark}, afterpulse probability p_{after}
- **Basis-dependent efficiency:** Different η for rectilinear vs. diagonal (for time-shift attacks)

8.3 Milestone 3: Seven Eavesdropping Attack Models

Implement `src/temporal_qber/attacks.py` with explicit configuration and random seed per attack:

1. **Intercept-Resend:** Eve measures each photon in random basis, resends measured state
2. **Photon Number Splitting (PNS):** Eve splits multi-photon pulses, stores one photon, forwards rest; measures stored photon after basis announcement
3. **Beam Splitting:** Eve taps fixed fraction of optical power, measures her portion
4. **Time-Shift:** Eve exploits basis-dependent detector efficiency by shifting arrival times
5. **Trojan Horse:** Eve injects bright light into Alice's/Bob's device, analyzes back-reflections
6. **Quantum Cloning:** Eve uses imperfect cloning machine (e.g., Bužek-Hillery), measures clone
7. **Fake State:** Eve sends prepared states to Bob, mimicking expected statistics
8. **Normal:** No eavesdropper (baseline channel)

Critical: Channel parameters (loss, noise, efficiency) must use **overlapping distributions** across all 8 classes to prevent label leakage (paper: "overlapping noise and loss distributions across normal and attack classes").

8.4 Milestone 4: Temporal QBER Extraction

Implement `src/temporal_qber/qber.py`:

- Window size $W = 50$ sifted bits (non-overlapping)
- QBER per window: $q_t = \frac{1}{W} \sum_{i=(t-1)W+1}^{tW} (a_i \oplus b_i)$
- Handle incomplete final windows: **drop** (documented policy)
- Minimum valid sifted key length: e.g., 200 bits (4 windows)
- Output: Sequence $[q_1, q_2, \ldots, q_T]$ per session

8.5 Milestone 5: 63 Physics-Informed Features

Extend `src/temporal_qber/features.py` (skeleton exists) with validated formulas for all 63 features:

1. **Statistical (7):** Mean, variance, std, skewness, kurtosis, IQR, CV
2. **Burst/Instability (15):** Spike density, burst duration (mean/max/total), jump energy, temporal drift, error-gap statistics (mean/std/CV/skew), consecutive error runs, max consecutive errors, clustering coefficient, instability index
3. **Spectral/Temporal (15):** Lag 1-3 autocorrelation, DFT energy (low/mid/high), spectral entropy, centroid, bandwidth, Shannon entropy, permutation entropy, sample entropy, Hurst exponent, Lyapunov exponent, correlation dimension
4. **Basis-Dependent (10):** Rectilinear QBER, diagonal QBER, basis difference, basis ratio, rectilinear burst energy, diagonal burst energy, basis asymmetry, basis switch rate, basis error correlation, basis drift
5. **Channel-Interaction (16):** Key generation rate, transmission efficiency, QBER-loss correlation, QBER-loss interaction, multi-scale QBER (scales 1,2,4,8,16,32), loss-burst correlation, detector efficiency, dark count rate, afterpulse probability, channel stability, QBER trend

Validation: Each feature as independently tested function with known inputs. Document every formula; paper only gives categories.

8.6 Milestone 6: Dataset Generation

Extend `src/temporal_qber/dataset.py`:

- Generate 3,000 sessions per class $\times$ 8 classes = 24,000 sessions
- Randomize channel conditions per session with overlapping distributions
- Extract 63 features per session
- Save: `session_id, f_001...f_063, label, metadata (seed, config)`
- Validate: No NaN/inf, balanced classes, reproducible with seed

8.7 Milestone 7: Training and Evaluation

Extend `src/temporal_qber/train.py`, `evaluate.py`:

- 80/20 stratified split, 5-fold CV on training set
- 10 independent seeds (full experiment repetition)
- Three classifiers: RF, XGBoost, SVM-RBF (same hyperparameters)
- Baseline comparison: Fixed 11% threshold, CUSUM detector on temporal QBER
- Binary detection metrics: Accuracy, FNR, FPR (attack vs. normal)
- Multiclass metrics: Accuracy, macro-F1, per-class F1, ROC-AUC (OvR)

8.8 Milestone 8: Explainability and Ablation

Extend `src/temporal_qber/explain.py`:

- SHAP on XGBoost (TreeExplainer) with temporal-QBER features
- Leave-one-feature-group-out ablation (5 groups: statistical, burst, spectral, basis, channel)
- Global SHAP summary, per-class dependence plots for top features

8.9 Milestone 9: Results Comparison

Target metrics from paper (10-seed average):

- XGBoost accuracy: 88.01%
- XGBoost macro-F1: 0.8803
- Binary detection: XGBoost 88.01% vs. CUSUM 78.99% vs. Threshold 25.82%

Note on exact reproduction: The paper's 6 pages do not fully specify all 63 feature formulas or every attack-simulation parameter. Our implementation will document all assumptions. Exact numerical reproduction requires the missing implementation details.

8.10 Implementation Dependencies

- No quantum framework required (classical probability simulation sufficient)
- Existing ML stack: scikit-learn, XGBoost, SHAP, NumPy, Pandas, SciPy
- Optional: Qiskit for validation against quantum simulations

9 Conclusion

This report presented a complete, reproducible machine-learning experiment for quantum attack detection using the Quantum Attack Dataset (QAD). Our key findings:

1. **High classification accuracy on QAD:** XGBoost achieves 95.8% accuracy and 0.947 macro-F1 on 8-class quantum circuit attack detection. Random Forest is comparable (95.2% / 0.939). SVM-RBF underperforms (72.2% / 0.705).
2. **Label leakage identified:** SHAP explainability and feature-group ablation conclusively show that performance is driven by attack-specific simulator parameters (`swap_intensity`, `dp_error_rate`, `mt_flip_prob`, etc.) that directly encode the attack type and intensity. Removing this group reduces accuracy by 15-22% for tree models.
3. **Not BB84 reproduction:** QAD's quantum-circuit attacks differ fundamentally from the paper's seven BB84 eavesdropping attacks. The temporal-QBER framework (windowed QBER sequences, 63 physics-informed features) is not implemented on QAD because QAD lacks temporal dimension and basis reconciliation.
4. **Reproducible pipeline:** All code, models, results, and dependencies are versioned and executable via single commands. The pipeline supports SHAP, ablation, and programmatic result access.

9.1 Path Forward

The roadmap for full BB84 temporal-QBER reproduction (Section 8) consists of 9 milestones:

1. BB84 simulator with basis reconciliation
2. Channel model (loss, noise, detector imperfections)
3. 7 eavesdropping attack models with overlapping channel parameters
4. Temporal QBER extraction (W=50 bits)
5. 63 validated physics-informed features
6. 24,000 balanced sessions generation
7. Training with 10-seed averaging and baseline comparison
8. SHAP and 5-group ablation on temporal features
9. Comparison to paper's 88.01% / 0.8803 targets

The existing codebase provides the ML infrastructure (train, evaluate, explain, predict) and feature extraction framework. The primary effort lies in the quantum simulation layer (Milestones 1-4) and feature validation (Milestone 5).

9.2 Final Remark

The QAD experiments serve as a valuable **infrastructure validation** for the ML pipeline, revealing the critical importance of preventing label leakage through careful feature design and channel parameter randomization—a lesson that directly informs the BB84 simulator design.

A Reproducibility

A.1 Package Versions

- Python 3.12.13
- NumPy 2.5.4
- Pandas 3.1.0
- scikit-learn 1.9.1
- XGBoost 3.4.1
- SHAP 0.54.1
- Matplotlib 3.11.2
- Seaborn 0.13.2
- Joblib 1.6.0
- SciPy 1.18.1

A.2 Experiment Configuration

- Dataset: QAD balanced, 10,000 samples (of 1,000,000 available)
- Features: 42 raw + 3 one-hot circuit_type = 45 model features
- Train/Test: 80/20 stratified, random_state=42
- CV: 5-fold stratified on training split only, random_state=42
- Class weights: balanced (Random Forest, SVM); sample weights for XGBoost
- Random Forest: 200 estimators, max_depth=20
- XGBoost: 200 estimators, max_depth=10, lr=0.1, sample weights from training labels
- SVM-RBF: C=1.0, gamma=scale, probability=True, class_weight=balanced

A.3 Saved Artifacts

- Models: `models/random_forest.joblib`, `models/xgboost.joblib`, `models/svm_rbf.joblib`
- Preprocessing: `models/preprocessing.joblib` (scaler + feature names)
- Class names: `models/class_names.joblib`
- Evaluation: `results/evaluation_results.json`
- Plots: `results/*.png`
- SHAP results: `results/*_shap_results.json`

A.4 Commands to Reproduce

```
# Full pipeline (retrain)
uv run python scripts/run_pipeline.py --n-samples 10000

# Quick test
uv run python scripts/test_pipeline.py

# Demo predictions
uv run python -m temporal_qber.predict

# Generate report
uv run python scripts/generate_report.py
```

References

1. **Bennett, C. H.** and **Brassard, G.** (1984). Quantum cryptography: Public key distribution and coin tossing. *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, 175–179.

2. **Shor, P. W.** and **Preskill, J.** (2000). Simple proof of security of the BB84 quantum key distribution protocol. *Physical Review Letters*, 85(2):441–444.

3. **Reference Paper** (2026). Beyond the QBER Threshold: A Temporal QBER Based Machine Learning Framework for Multi Attack Detection in BB84 QKD. *arXiv preprint arXiv:2608.04047v1*.

4. **Udugahapattuwa, P.**, **de Alwis, C.**, **Wijewardane, U.**, and **Liyanage, M.** (2026). A Comprehensive Quantum Attack Dataset (QAD). *Kaggle*. DOI: 10.34740/kaggle/ds/9778273. URL: Kaggle dataset page.

5. **Pedregosa, F.**, **Varoquaux, G.**, **Gramfort, A.**, et al. (2011). scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

6. **Chen, T.** and **Guestrin, C.** (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

7. **Lundberg, S. M.** and **Lee, S.-I.** (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30:4765–4774.